**PONDICHERRY UNIVERSITY**
**RAMANUJAN SCHOOL OF MATHEMATICAL SCIENCES**
**DEPARTMENT OF STATISTICS**

**M.Sc. QUANTITATIVE FINANCE**

**MSQF 534: Corporate Internship**

**REPORT ON**

# FinMetrics Suite

An AI-Powered Cross-Sectional Financial Analysis, Anomaly Detection, and Strategic Benchmarking System

**SUBMITTED BY**

**NAME: N Rohit Vedhanandh**

**REG. NO.: 25MSQUFPY0002**

**NOVEMBER 2026**

# PONDICHERRY UNIVERSITY
# RAMANUJAN SCHOOL OF MATHEMATICAL SCIENCES
# DEPARTMENT OF STATISTICS

## CERTIFICATE

Certified to be the bonafide record of work done in **MSQF 534: Corporate Internship** by **N Rohit Vedhanandh** with Register No. **25MSQUFPY0002** of class **II M.Sc. Quantitative Finance** in the Department of Statistics, Pondicherry University, Puducherry 605 014, during the academic year 2026-27.

_____ _____

**Faculty - in - charge** **Coordinator**

**Date:**

**Submitted for the M.Sc. Degree Practical Examination held on _____.**

_____

**Examiners**

IQBA
Business Advisors

# Certificate
## OF INTERNSHIP

This certificate is proudly presented to

## N. Rohit Vedhanandh

has successfully completed a 2-month internship at **IQ Business Advisors** from **06th May, 2026 to 10th July, 2026**. During this period, he actively contributed to tasks related to **Financial Analyst and Process Re-engineering**, demonstrating a strong willingness to learn and a professional attitude. His contributions were appreciated by the team, and he consistently showed initiative and responsibility in the assigned work. We thank for his time and efforts and wish his success in future endeavors.

**Issue Date** : 10/07/2026
**Certificate No** : IQBA/2026/013

IQ BUSINESS ADVISORS

**N NAGA ARJUN**
**Managing Partner**

# ABSTRACT

Automated cross-sectional financial analysis is a critical tool in modern corporate intelligence and equity research, yet it remains constrained by reporting nomenclature disparities and baseline outlier skewness across public market filings. This thesis presents the design, implementation, and evaluation of the ***FinMetrics Suite*** (the *Financial Outlier Detection and Strategic Benchmarking System*), an end-to-end Python-based analytical framework. The system features automated public financial statement harvesting via the Yahoo Finance API, a custom standardization parser with a 60-field alias map to resolve naming variations in public disclosures, a robust statistical engine implementing Tukey Fences (IQR) and robust Modified Z-Scores (using Median Absolute Deviation) to identify and mask reporting anomalies, and an unsupervised multivariate anomaly detector using Isolation Forests. To automate peer group alignment, the framework integrates the Gemini AI model to dynamically search, discover, and define the correct market sector for a given ticker or abstract query.

To evaluate company financial standing against robust industry baselines, the framework introduces a 5-pillar composite scoring model that maps standardized financial ratios across Profitability, Liquidity, Solvency, Efficiency, and Growth to normalized 0–100 scores using a bounded Sigmoid transformation. Empirical evaluation across 13 National Stock Exchange (NSE) sectors demonstrates that our outlier masking routine prevents baseline distortion, reducing the standard deviation of an outlier-contaminated sector Operating Margin distribution from 39.54% to 1.28%. The system outputs interactive visual dashboards, stateful Gemini-driven diagnostic chat sessions, and professionally formatted multi-sheet Excel reports, providing quantitative analysts, equity researchers, and financial advisors with institutional-grade competitive insights.

# Contents

**Certificate** **i**

**Internship Certificate** **ii**

**Abstract** **iii**

**Table of Contents** **iv**

**List of Abbreviations** **vii**

**1 Introduction** **1**

1.1 Background . . . . . 1

1.2 Problem Statement . . . . . 1

1.3 Objectives . . . . . 2

1.4 Scope of the Project . . . . . 3

1.5 Motivation . . . . . 3

**2 Literature Survey / Related Work** **4**

2.1 Existing Methodologies . . . . . 4

2.2 Machine Learning & Robust Outlier Detection in Finance . . . . . 4

2.3 Research Gaps . . . . . 5

**3 System Analysis and Design** **7**

3.1 System Architecture . . . . . 7

3.2 System Workflow and Execution Pipeline . . . . . 8

3.3 Data Flow Diagrams . . . . . 10

3.3.1 DFD Level 0: Context Diagram . . . . . 10

3.3.2 DFD Level 1: Process Diagram . . . . . 10

3.4 Module Descriptions . . . . . 11

**4 Algorithms and Methodology** **13**

4.1 Tukey Fences (IQR Outlier Detection) . . . . . 13

4.2 Robust Modified Z-Score . . . . . 14

4.3 Isolation Forest (Multivariate Anomaly Detection) . . . . . 15

4.4 Algorithms Pseudocode . . . . . 17

4.4.1 Univariate Outlier Detection and Baseline Masking . . . . . 17

**5 Implementation** **19**

5.1 Technologies Used . . . . . 19

5.2 Tools and Libraries . . . . . 19

5.3 Development Workflow and Methodology . . . . . 20

5.4 Code Structure Explanation . . . . . 20

5.5 Module-wise Implementation Details . . . . . 21

5.5.1 Financial Standardizer (financial_parser.py) . . . . . 21

5.5.2 Outlier Scoring (outlier_engine.py) . . . . . 22

5.6 System Component Integration . . . . . 22

**6 Results and Discussion** **24**

6.1 Experimental Setup . . . . . 24

6.2 Results Analysis . . . . . 24

6.3 Outlier Masking Evaluation . . . . . 25

6.4 Benchmarking Case Study: Listed Enterprise Evaluation . . . . . 25

6.4.1 Strategic Diagnostic Insights . . . . . 26

6.5 Performance Analysis . . . . . 27

**7 Conclusion and Future Enhancements** **28**

7.1 Final Outcomes . . . . . 28

7.2 Contributions . . . . . 28
7.3 Future Scope . . . . . 29

**Bibliography** **30**

**A Core Source Code** **31**

A.1 Outlier Filtering Logic . . . . . 31

# List of Abbreviations

| Abbreviation | Full Form |
|---|---|
| **API** | Application Programming Interface |
| **CAGR** | Compound Annual Growth Rate |
| **COGS** | Cost of Goods Sold |
| **DFD** | Data Flow Diagram |
| **EBIT** | Earnings Before Interest and Taxes |
| **EBITDA** | Earnings Before Interest, Taxes, Depreciation, and Amortization |
| **EPS** | Earnings Per Share |
| **FP&A** | Financial Planning and Analysis |
| **IQR** | Interquartile Range |
| **MAD** | Median Absolute Deviation |
| **ML** | Machine Learning |
| **OCF** | Operating Cash Flow |
| **QoQ** | Quarter-on-Quarter |
| **ROA** | Return on Assets |
| **ROCE** | Return on Capital Employed |
| **ROE** | Return on Equity |
| **SWOT** | Strengths, Weaknesses, Opportunities, and Threats |
| **TTL** | Time-to-Live |
| **UML** | Unified Modeling Language |
| **YoY** | Year-on-Year |

# Chapter 1

# Introduction

## 1.1 Background

In contemporary financial markets, the ability to conduct fast, precise, and objective cross-sectional financial analysis is paramount for investment professionals, credit risk analysts, and corporate managers. Traditional financial statement analysis relies on the examination of a firm's three core sheets: the Income Statement, the Balance Sheet, and the Cash Flow Statement. Historically, this process was heavily manual, requiring analysts to extract figures from PDF filings, input them into customized spreadsheet models, and perform comparison arithmetic manually.

However, with the rapid digitization of financial reporting, modern quantitative finance demands automated pipelines. The proliferation of web services, high-throughput APIs, and advanced machine learning models has transformed corporate intelligence. Analysts are no longer satisfied with simple historical ratio sheets; they require statistical models that can process entire peer groups, identify anomalous reporting structures, rank corporate financial health objectively, and benchmark company financial health objectively against robust industry baselines in real time.

## 1.2 Problem Statement

Automated cross-sectional financial analysis is constrained by two primary technical bottlenecks:

1. **Nomenclature Disparity**: Different corporate entities, regulatory bodies, and geographical systems list identical accounting items under a wide variety of names. For example, top-line sales can be reported as *"Total Revenue"*, *"Revenue"*, *"Net Sales"*, or *"Operating Revenue"*. Similarly, operating profit can be reported as *"EBIT"*, *"Operating Income"*, or *"Operating Profit"*. This lack of standardization breaks standard web-scraping scripts and automated database integrations.

2. **Outlier Distortion**: Financial datasets frequently contain statistical anomalies due to one-off events, corporate restructurings, asset impairment write-downs, or extreme debt-leveraging actions. If left untreated, these outliers heavily distort traditional comparative baselines (such as the peer mean and standard deviation). A single distressed company or an exceptionally profitable pure-play firm can bias the sector average, making standard comparative rankings mathematically invalid.

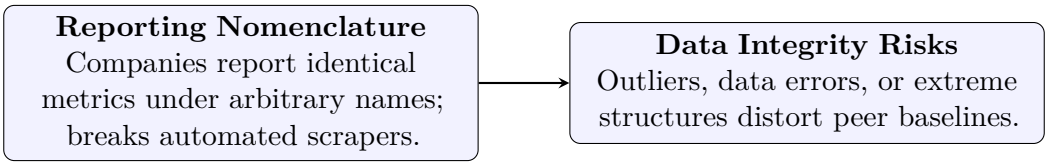


Figure 1.1: Core Bottlenecks in Automated Financial Benchmarking

## 1.3 Objectives

The primary objectives of the *FinMetrics Suite* (or the *Financial Outlier Detection and Strategic Benchmarking System*) are:

- **Automated Financial Harvesting**: Designing a high-performance, cached ingestion pipeline that fetches real-time quarterly and annual statements for major publicly listed companies using the Yahoo Finance API.
- **Standardized Alias Processing**: Implementing a robust financial parser equipped with a 60-field canonical mapping dictionary to automatically resolve nomenclature variance in public filings.
- **Robust Anomaly Scanning**: Constructing a statistical and machine learning kernel that runs Tukey Fences (IQR), robust Modified Z-Scores (using Median Absolute Deviation), and unsupervised Isolation Forests to identify and mask reporting outliers.
- **Weighted Pillar Scoring**: Developing a 5-pillar composite health framework (Profitability, Solvency, Growth, Liquidity, Efficiency) to objectively grade and rank peer groups.
- **Strategic Benchmarking Diagnostics**: Auto-synthesizing relative percentile ranks into a 2x2 SWOT grid and initiating an interactive consulting dialogue with Gemini.

## 1.4 Scope of the Project

The technical scope of the project encompasses:

- A Python-based analytical package containing isolated libraries for fetching, parsing, modeling, and exporting data.
- A Streamlit web application that serves as the visual control dashboard.
- Coverage of 13 primary public market sectors (e.g., Automobiles, IT, FMCG, Banking, Pharmaceuticals) representing over 100 publicly listed firms.
- Integration of the Gemini AI model to dynamically search, discover, and define the corresponding sector or market for a given ticker or abstract query.
- Direct export capabilities to styled multi-sheet Excel workbooks and CSV files.

The project is designed as a decision-support platform for financial analysts, equity researchers, and corporate advisors. It does not execute automated trading orders or make unsupervised capital allocation investments.

## 1.5 Motivation

The core motivation for this research stems from the democratization of corporate finance. While large institutional investment banks possess expensive proprietary databases (such as Bloomberg Terminals or Reuters Eikon) and legions of junior analysts to parse financial statements, small-scale equity researchers and independent financial advisors are locked out of this ecosystem due to massive subscription costs.

By building an open-source, high-efficiency system that automates public data collection, standardizes reporting nomenclature, and applies institutional-grade quantitative anomaly models, we can provide professional-grade competitive intelligence to equity analysts and financial researchers. Analysts can rapidly discover cross-sectional peer groups, filter reporting distortions through robust statistical baselines, and immediately evaluate how a company's working capital velocity, pricing power, and leverage profile stack up against listed sector champions. This eliminates manual spreadsheet friction and empowers financial professionals with rigorous, data-driven decision tools.

# Chapter 2

# Literature Survey / Related Work

## 2.1 Existing Methodologies

Traditional corporate benchmarking has evolved from manual spreadsheet transcription to proprietary financial databases:

- **Manual Spreadsheet Benchmarking**: Analysts manually extract and transcribe quarterly figures into Microsoft Excel models. This foundational framework, formalized by Damodaran [1], remains slow, labor-intensive, and prone to human error.
- **Proprietary Financial Terminals**: Bloomberg Terminals and S&P Capital IQ automate data harvesting and standardization. However, their high annual licensing fees ($24,000+ per user) and closed database architectures prevent independent researchers from easily integrating custom statistical outlier screening models.

## 2.2 Machine Learning & Robust Outlier Detection in Finance

Corporate financial datasets frequently contain extreme outliers from non-recurring activities, distorting standard mean-based Z-score benchmarking. To resolve this, robust statistics and unsupervised machine learning models are deployed:

- **Tukey's Fences (IQR)**: Non-parametric filter using the Interquartile Range ($IQR = Q_3 - Q_1$) to define bounds as $[Q_1 - k \cdot IQR,\ Q_3 + k \cdot IQR]$ [3]. Resilient against extreme skewness as it relies on medians.
- **Modified Z-Scores (MAD)**: Measures statistical dispersion via the Median Absolute Deviation ($MAD = \text{median}(|x_i - \tilde{x}|)$). The Modified Z-Score $M_i$ is formulated as $M_i = 0.6745(x_i - \tilde{x})/MAD$. Flags outliers exceeding 3.5 robustly.

- **Isolation Forests (iForest)**: An unsupervised machine learning algorithm [2] that isolates anomalies in multi-dimensional space by randomly partitioning feature values. Anomalies isolate closer to the tree root, resulting in shorter path lengths.

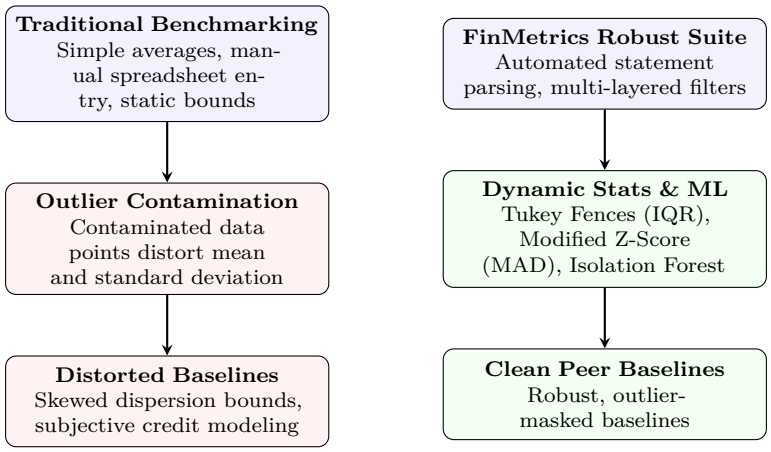


Figure 2.1: Conceptual Comparison of Traditional vs. Robust Machine Learning Benchmarking Pipelines

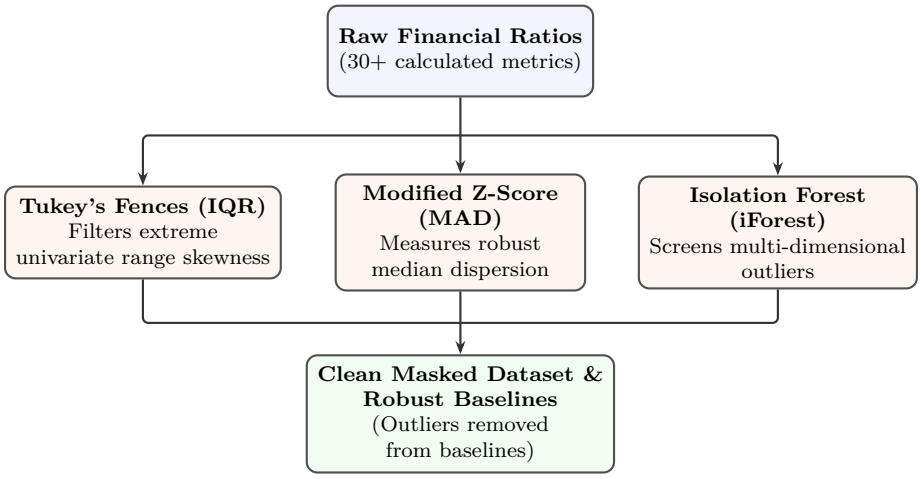


Figure 2.2: Multi-layered Robust Outlier Masking and Baseline Generation Pipeline

## 2.3 Research Gaps

Two critical research gaps remain in current financial intelligence frameworks:

- **Lack of Nomenclature Autonomy**: Scrapers fail to handle inconsistent reporting names, causing column mismatches or missing values in raw statement parsing.

- **Baseline Outlier Contamination**: Distressed peer data distorts average standard deviations. Standard methods lack automatic statistical masking before baseline generation.

# Chapter 3

# System Analysis and Design

## 3.1 System Architecture

The *FinMetrics Suite* is structured as a decoupled, multi-layer architecture. This decoupling ensures that the user interface (UI) layer is separated from the analytical modules, database cache, and API engines, facilitating code maintenance and extensions.

The architecture is divided into three primary layers:

1. **Presentation Layer (Streamlit UI)**: Renders the sidebar controls (sector/scope selection, threshold calibrations), interactive analytical tabs, Plotly charts, and styled report export facilities.
2. **Analytical Kernel Layer (Python Packages)**: The core processing layer containing standard modules:
    - **Sector Loader**: Maps sectors to public tickers.
    - **Data Fetcher**: Retrieves Yahoo Finance statements and manages local cache.
    - **Financial Parser**: Standardizes columns using the alias dictionary.
    - **Ratio Engine**: Computes 30+ ratios and YoY growth rates across 5 core pillars.
    - **Outlier Engine**: Identifies anomalies using multi-tier statistical filters (IQR, MAD Z-score, Isolation Forest).
    - **Benchmark Core**: Coordinates baseline calculation, cell-level outlier masking, and robust peer statistics.
    - **Benchmark Classifier**: Assigns health labels, signal classes, and percentile rankings.
    - **Benchmark Store**: Persists and retrieves precomputed sector benchmark matrices in Parquet format.
    - **AI Handler**: Manages Gemini ticker discovery and diagnostic commentary.

- **Report Generator**: Compiles multi-sheet styled Excel workbooks and analytical outputs.

3. **Data Cache Layer**: Persistent storage in JSON and Parquet formats to avoid API rate limits.

## 3.2 System Workflow and Execution Pipeline

The application executes in a clean, stateful single-track process flow comprising ten sequential execution stages:

1. **Scope Definition**: The user defines the benchmark scope via the sidebar by selecting a pre-mapped Sector, choosing a granular Industry, or inputting an abstract search prompt for Gemini AI Search.

2. **Environment Check & API Key Configuration**: The sidebar inspects environment variables and user configuration to verify active Google Gemini API keys and runtime parameters.

3. **AI Ticker Discovery**: If AI Search is selected, the *AI Handler* queries Gemini to discover relevant listed stock tickers dynamically, appending required exchange suffixes (e.g., `.NS` for the National Stock Exchange of India).

4. **Statement Harvesting & Caching**: The *Data Fetcher* checks local storage for cached statements; if missing or expired, it queries the Yahoo Finance API via `data_fetcher.py` and stores normalized statements in a 24-hour Parquet cache.

5. **Canonical Synonym Resolution**: The *Financial Parser* (`financial_parser.py`) resolves nomenclature variations across accounting reporting formats using a 60-field canonical alias map (`ALIAS_MAP`), standardizing statements into uniform DataFrames.

6. **Ratio Calculation**: The *Ratio Engine* (`ratio_engine.py`) computes over 30 standardized financial ratios and YoY growth metrics organized across 5 core pillars: Liquidity, Solvency, Profitability, Efficiency, and Valuation.

7. **Multi-Tier Outlier Detection**: The *Outlier Engine* (`outlier_engine.py`) executes statistical anomaly screening across all ratios using an ensemble of Interquartile Range (IQR) fences, MAD-based modified Z-scores, and multivariate Isolation Forests.

8. **Outlier Masking and Baseline Generation**: The *Benchmark Core* (`benchmark_core.py`) applies cell-level masking to flagged statistical anomalies and computes robust peer statistics (medians, interquartile spreads, and deciles).

9. **Scoring, Aggregation and Ranking**: The *Benchmark Classifier* (`benchmark_classifier.py`) applies non-linear sigmoid transformations to convert metric deviations into bounded scores (0–100), computes weighted pillar aggregations, and determines final percentile rankings.

10. **Output Generation**: The system renders interactive Streamlit tabs, generates styled multi-sheet Excel workbooks via `report_generator.py`, and triggers Gemini-powered diagnostic commentary.

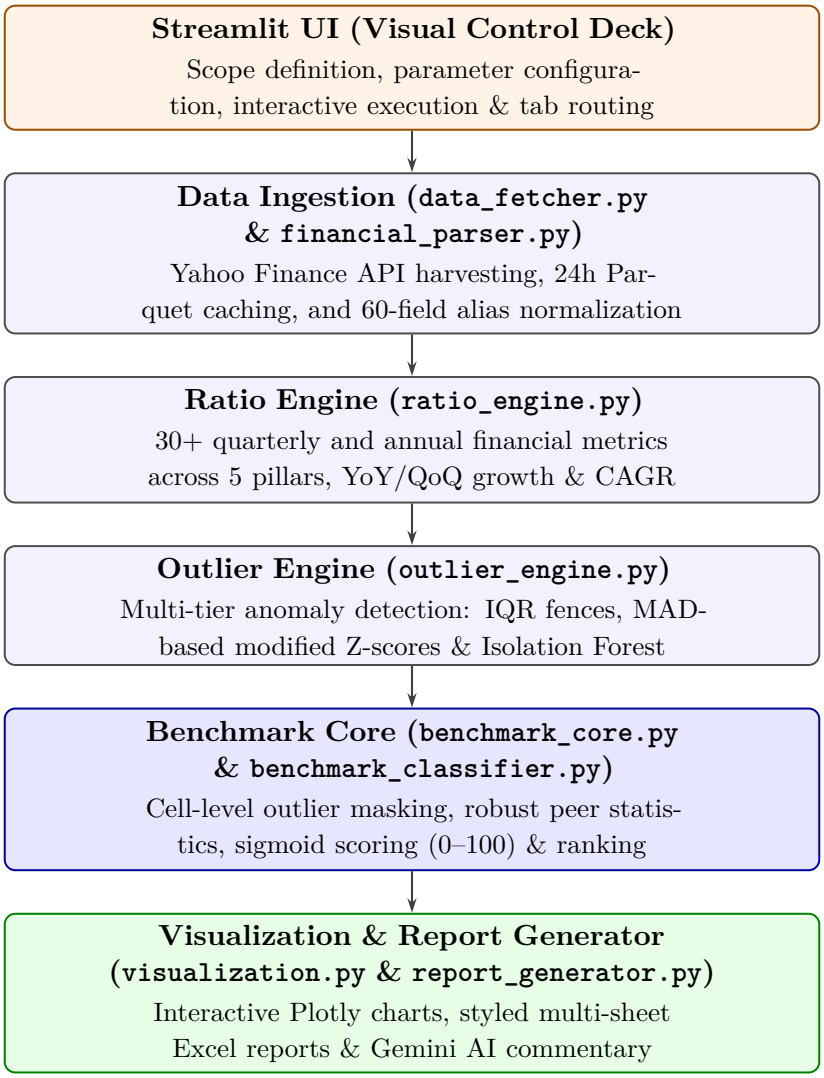


Figure 3.1: End-to-End System Architecture and Processing Pipeline

## 3.3 Data Flow Diagrams

To detail the data movement within the platform, DFD Level 0 (Context Diagram) and DFD Level 1 (Process Diagram) are provided below.

### 3.3.1 DFD Level 0: Context Diagram

The Level 0 Data Flow Diagram (Figure 3.2) outlines the primary external entities interacting with the FinMetrics Suite.

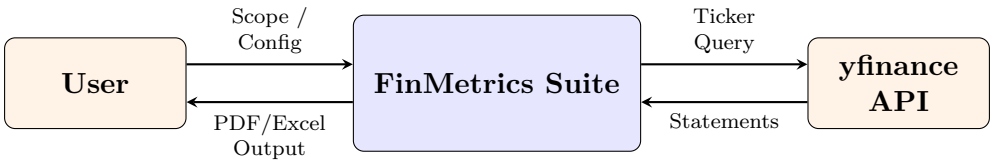


Figure 3.2: Data Flow Diagram Level 0: Context Diagram

### 3.3.2 DFD Level 1: Process Diagram

The Level 1 Data Flow Diagram (Figure 3.3) details the internal processes, data repositories, and transitions.

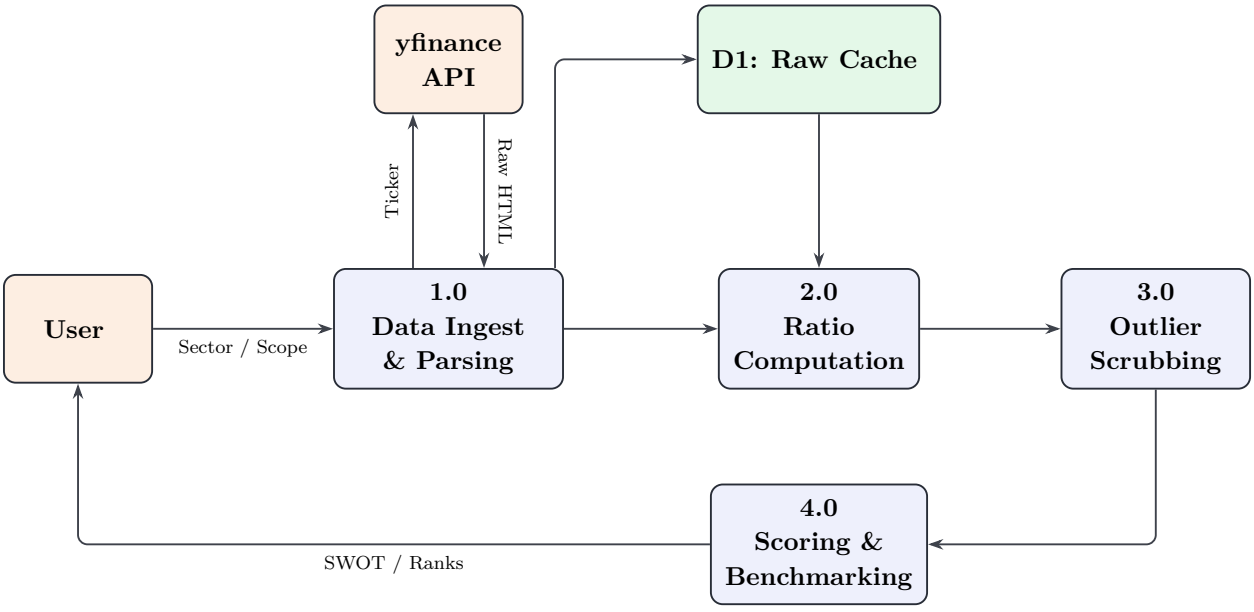


Figure 3.3: Data Flow Diagram Level 1: Process Diagram

## 3.4 Module Descriptions

The system consists of the following modules, each encapsulating specific functional logic:

- **sector_loader.py**: A template mapping utility that lists standard public market tickers under 13 sectors: Automobiles, IT, FMCG, Banking, Pharma, Oil & Gas, Metals, Cement, Telecom, Power, Realty, Capital Goods, and Infrastructure.
- **data_fetcher.py**: Handles secure connections to the ‘yfinance‘ Ticker wrapper. Implements asynchronous API requests and caches raw quarterly/annual statements in Parquet format with a 24-hour TTL config.
- **financial_parser.py**: Normalizes messy statements into clean, structured DataFrames. Scans the rows, renames matched variants using the alias map `ALIAS_MAP`, removes duplicate metrics, and converts all entries to numeric types.
- **ratio_engine.py**: Performs financial ratio computations. Reconstructs quarterly time-series, calculates YoY/QoQ percentage changes, and computes CAGR growth trends across 5 core pillars.
- **outlier_engine.py**: The statistical processing module. Implements multi-tier outlier detection via IQR fences, MAD-based modified Z-scores, and multivariate Isolation Forest.
- **benchmark_core.py**: Orchestrates the baseline calculation. Masks outlier cells, computes clean peer statistics (medians, deciles), and ranks firms by their weighted composite health score.
- **benchmark_classifier.py**: The business logic translator. Assigns health labels, signal classes, and percentile ranks based on metric performance against sector distributions.
- **benchmark_store.py**: Persists and retrieves precomputed sector benchmark matrices in Parquet format.
- **report_generator.py**: Generates styled Excel workbooks using custom colors, column auto-fitting, and formatted dashboards.
- **ai_handler.py**: Integrates with the Gemini API to search for sector tickers and run the interactive diagnostic chat.

- **visualization.py**: Renders interactive charts (radar profiles, distributions, stacked contribution bars, heatmaps) in Plotly and Seaborn.

# Chapter 4

# Algorithms and Methodology

## 4.1 Tukey Fences (IQR Outlier Detection)

The Interquartile Range (IQR) represents a robust statistical measure of dispersion. Unlike the standard deviation, the IQR is highly resistant to extreme outliers because it is computed from the middle 50% of the distribution.

The IQR is defined as the difference between the third quartile ($Q_3$, 75th percentile) and the first quartile ($Q_1$, 25th percentile):

$$\text{IQR} = Q_3 - Q_1 \tag{4.1}$$

Tukey Fences establish the lower and upper bounds of the acceptable financial distribution using an outlier multiplier ($k$, typically set to 1.50):

$$\text{Lower Bound} = Q_1 - k \times \text{IQR} \tag{4.2}$$

$$\text{Upper Bound} = Q_3 + k \times \text{IQR} \tag{4.3}$$

To formalize this outlier screening as a statistical decision process, we define the following hypothesis test for each corporate observation in the peer group:

$$\mathbf{H}_0 : \textbf{The observation is a normal inlier} \tag{4.4}$$

$$\mathbf{H}_1 : \textbf{The observation is an outlier} \tag{4.5}$$

Under the null hypothesis, the observation is assumed to lie within the normal statistical boundaries of the peer group. If the observation $x_i$ (representing the financial ratio of company $i$) falls outside the Tukey Fences, we reject $H_0$ in favor of $H_1$, and the cell is flagged as a univariate outlier:

$$x_i < \text{Lower Bound} \quad \text{or} \quad x_i > \text{Upper Bound} \tag{4.6}$$

To provide a quick operational reference, the primary variables utilized in Tukey outlier boundaries are defined in Table 4.1.

Table 4.1: Tukey Fences Outlier Detection Variables

| Variable / Symbol | Operational Definition and Description |
|---|---|
| **IQR** | Interquartile Range, measuring the dispersion of the middle 50% of peers. |
| $\mathbf{Q_1}, \mathbf{Q_3}$ | The first (25th percentile) and third (75th percentile) peer quartiles. |
| **k** | Outlier multiplier parameter (default 1.50) governing Tukey fence width. |
| $\mathbf{x}_i$ | Evaluated financial ratio value for a specific corporate entity $i$. |

## 4.2 Robust Modified Z-Score

While standard Z-scores rely on the mean and standard deviation, they are highly sensitive to extreme outliers. To address this, we implement a robust Modified Z-score that uses the median and the Median Absolute Deviation (MAD).

First, the Median Absolute Deviation (MAD) is calculated:

$$\text{MAD} = \text{median}\left(|x_i - \tilde{x}|\right) \tag{4.7}$$

Where $\tilde{x}$ is the median of the peer group. The Modified Z-score ($z_i$) for each company $i$ is defined as:

$$z_i = 0.6745 \times \frac{x_i - \tilde{x}}{\text{MAD}} \tag{4.8}$$

The constant 0.6745 serves as a scale factor, ensuring that the expected value of the robust Z-score matches standard normal Z-scores for normally distributed datasets.

To evaluate the standardized deviation of each corporate observation, the outlier screening is formulated as a hypothesis test:

$$\mathbf{H}_0 : \textbf{The deviation from the median is within normal variation} \tag{4.9}$$

$$\mathbf{H}_1 : \textbf{The deviation from the median is statistically extreme} \tag{4.10}$$

We fail to reject $H_0$ for normal observations (inliers). If the absolute modified Z-score $|z_i|$

exceeds the critical threshold $\theta_{\text{zscore}}$, the null hypothesis is rejected, and the ratio is flagged as a univariate outlier:

$$|z_i| > \theta_{\text{zscore}} \tag{4.11}$$

Where $\theta_{\text{zscore}}$ is the threshold parameter (default 3.50).

The mathematical symbols and operational parameters for robust Modified Z-score calculation are defined in Table 4.2.

Table 4.2: Robust Modified Z-Score Variables

| Variable / Symbol | Operational Definition and Description |
|---|---|
| $\tilde{\boldsymbol{x}}$ | Median of the corporate peer group for the evaluated financial ratio. |
| **MAD** | Median Absolute Deviation, a robust outlier-resistant dispersion measure. |
| $\mathbf{z}_i$ | Robust modified Z-score calculated for company $i$ using median and MAD. |
| $\boldsymbol{\theta}_{\textbf{zscore}}$ | Severity threshold (default 3.50) for robust Z-score outlier flagging. |

## 4.3 Isolation Forest (Multivariate Anomaly Detection)

Univariate outlier detection methods evaluate each ratio independently. However, financial reporting irregularities often manifest as multi-dimensional structural anomalies across multiple ratios simultaneously.

The Isolation Forest algorithm isolates anomalies by randomly partitioning high-dimensional feature spaces. Because anomalous points require fewer partitions to isolate, they appear closer to the root of the decision trees.

The anomaly score ($s$) for a sample $x$ is defined as:

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}} \tag{4.12}$$

Where $h(x)$ is the path length to isolate sample $x$, $E(h(x))$ is the average path length across all trees, and $c(n)$ is the average path length of an unsuccessful search in a Binary Search Tree:

$$c(n) = 2\ln(n-1) + 0.5772156649 - \frac{2(n-1)}{n} \tag{4.13}$$

An anomaly score close to 1.0 indicates a highly anomalous structural profile across all

financial dimensions.

To classify corporate profiles based on joint financial ratios, we formulate multivariate anomaly detection as a decision-theoretic hypothesis test:

$$\mathbf{H}_0 : \textbf{The sample exhibits a typical joint distribution across features} \tag{4.14}$$

$$\mathbf{H}_1 : \textbf{The sample exhibits a structurally anomalous joint distribution} \tag{4.15}$$

Where $\tau$ is the critical decision threshold (set to 0.5 by default, which signifies that the expected path length matches the average path length of an unsuccessful search in a binary search tree). Under $H_0$, the sample is classified as a normal corporate profile. If the anomaly score is greater than or equal to $\tau$, the null hypothesis is rejected, and the corporate entity is classified as a multivariate anomaly.

The mathematical parameters governing the tree-isolation metrics are summarized in Table 4.3.

Table 4.3: Isolation Forest Anomaly Detection Variables

| Variable / Symbol | Operational Definition and Description |
|---|---|
| $\mathbf{s}(\mathbf{x}, \mathbf{n})$ | Anomaly score for sample $x$ across the decision tree ensemble. |
| $\mathbf{n}$ | Total number of corporate samples / peer group size for tree partitioning. |
| $\mathbf{h}(\mathbf{x})$ | Traversed path length (number of edges) to isolate sample $x$ in a tree. |
| $\mathbf{E}(\mathbf{h}(\mathbf{x}))$ | Expected path length $h(x)$ across the entire Isolation Forest ensemble. |
| $\mathbf{c}(\mathbf{n})$ | Average path length of an unsuccessful search in a BST of size $n$. |

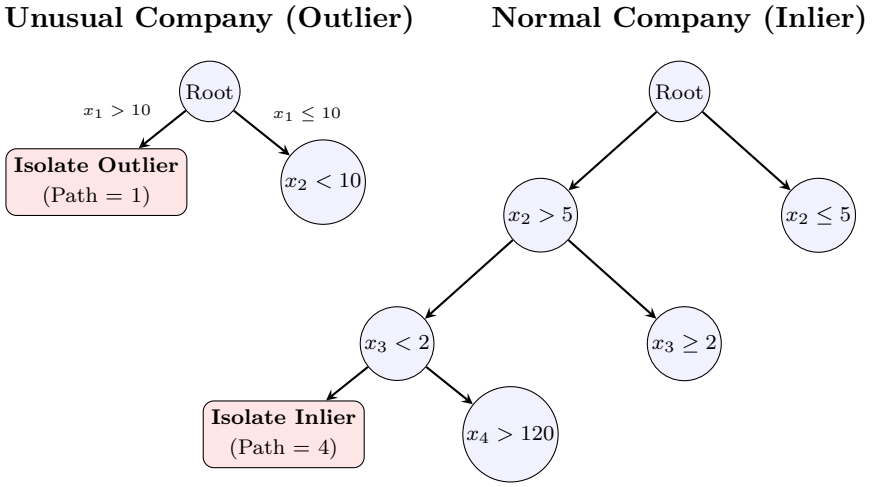


Figure 4.1: Multivariate Anomaly Isolation via Partition Tree Path Lengths

## 4.4 Algorithms Pseudocode

To facilitate the practical deployment and computational replication of our analytical framework, this section formalizes the core processes as structured, language-agnostic pseudocode. Specifically, we present the algorithmic logic governing our univariate outlier detection and baseline masking engine, which maintains statistical integrity across noisy peer distributions. To facilitate cross-referencing and trace the exact operations, the primary variables and notations deployed across the pseudocode are defined in Table 4.4.

Table 4.4: Algorithms Pseudocode Mathematical Symbols

| Variable / Symbol | Operational Definition and Description |
|---|---|
| $\mathbf{R}, \mathbf{C}$ | Input ratio matrix ($N \times M$) and output cleaned/masked ratio matrix. |
| $\mathbf{O}$ | Outlier database tracking tagged anomalies in $\{(company, ratio, value)\}$. |
| $\mathbf{V}, \mathbf{Z}$ | Non-null value vector and computed modified Z-scores for a ratio column. |
| $\mathbf{LB}, \mathbf{UB}$ | Calculated Tukey outlier boundaries (Lower Bound and Upper Bound). |

### 4.4.1 Univariate Outlier Detection and Baseline Masking

The outlier detection procedure (detailed in Algorithm 1) robustly identifies and isolates anomalies in financial ratio distributions. The algorithm operates as follows:

1. **Data Cleaning**: For each financial ratio column, it drops any missing values to extract a clean vector $V$.

2. **Statistical Thresholding**: It calculates the Tukey IQR fences (Lower Bound $LB$, Upper Bound $UB$) and robust Modified Z-scores based on the Median Absolute Deviation (MAD) of the distribution.

3. **Univariate Anomaly Flagging**: For each company, it checks if its ratio value $v$ falls outside the Tukey fences (iqr_flag) or if its Modified Z-score exceeds the threshold (z_flag).

4. **Outlier Masking**: If an anomaly is detected, the company is added to the outlier database $O$, and the clean copy matrix $C$ masks this specific cell as NaN (Not a Number). This prevents contaminated entries from skewing the peer group median and dispersion baselines.

**Algorithm 1** Univariate Outlier Detection and Baselines

**procedure** DetectOutliers($R, k, \theta$)
  $O \leftarrow \emptyset$; $C \leftarrow R$ ▷ *Initialize outlier set and clean copy*
  **for** each ratio column $j$ in $R$ **do**
    $V \leftarrow \text{dropna}(R[:, j])$
    **if** $\text{length}(V) < 3$ **then continue**
    $Q_1, Q_3 \leftarrow \text{percentile}(V, 25), \text{percentile}(V, 75)$
    $\text{IQR} \leftarrow Q_3 - Q_1$
    $\text{LB}, \text{UB} \leftarrow Q_1 - k \cdot \text{IQR}, Q_3 + k \cdot \text{IQR}$
    $\tilde{v} \leftarrow \text{median}(V)$; $\text{MAD} \leftarrow \text{median}(|V - \tilde{v}|)$
    **if** $\text{MAD} = 0$ **then**
      $\sigma \leftarrow \text{std}(V)$; $Z \leftarrow (V - \text{mean}(V))/\sigma$
    **else**
      $Z \leftarrow 0.6745 \times (V - \tilde{v})/\text{MAD}$
    **end if**
    **for** each company index $i$ in $R$ **do**
      $v \leftarrow R[i, j]$
      $\text{iqr_flag} \leftarrow (v < \text{LB}) \vee (v > \text{UB})$; $\text{z_flag} \leftarrow |Z[i]| > \theta$
      **if** $\text{iqr_flag} \vee \text{z_flag}$ **then**
        $O \leftarrow O \cup \{(i, j, v)\}$; $C[i, j] \leftarrow \text{NaN}$ ▷ *Mask outlier cell*
      **end if**
    **end for**
  **end for**
  **return** $O, C$
**end procedure**

# Chapter 5

# Implementation

## 5.1 Technologies Used

The *FinMetrics Suite* is developed entirely in Python 3.10+, utilizing standard scientific computing, data science, and machine learning libraries.

## 5.2 Tools and Libraries

The package dependencies and their structural roles are defined below:

- **Pandas (v2.1.1)**: Used for high-speed tabular operations and statement transpositions.
- **NumPy (v1.26.0)**: Provides vector operations, array manipulation, and robust calculations to avoid zero-division errors.
- **SciPy (v1.11.3)**: Supplies scientific functions, specifically ‘scipy.stats.percentileofscore‘ for ranking companies.
- **Scikit-Learn (v1.3.1)**: Powers the machine learning engines: RobustScaler for median-based scaling and IsolationForest for multi-dimensional anomaly detection.
- **Streamlit (v1.27.2)**: Provides the web frontend framework, handling interactive dashboards, stateful re-runs, and file upload structures.
- **Plotly (v5.17.0)**: Generates interactive visualizations (scatter plots, rank bar charts, heatmaps).
- **yfinance (v0.2.31)**: Handles connections to the Yahoo Finance API to fetch statements and market metrics.
- **openpyxl & XlsxWriter (v3.1.2)**: Generates styled multi-sheet Excel reports.
- **google-generativeai (v0.3.1)**: Manages Gemini model integration for natural-language stock discovery and diagnostic chats.

## 5.3 Development Workflow and Methodology

The system was designed and developed following an iterative, modular development lifecycle. Decoupling the data harvesters, standardizing parsers, and statistical scoring kernels allowed for thorough testing and validation before building the frontend app layer. The sequential phases of our development workflow are illustrated in Figure 5.1.

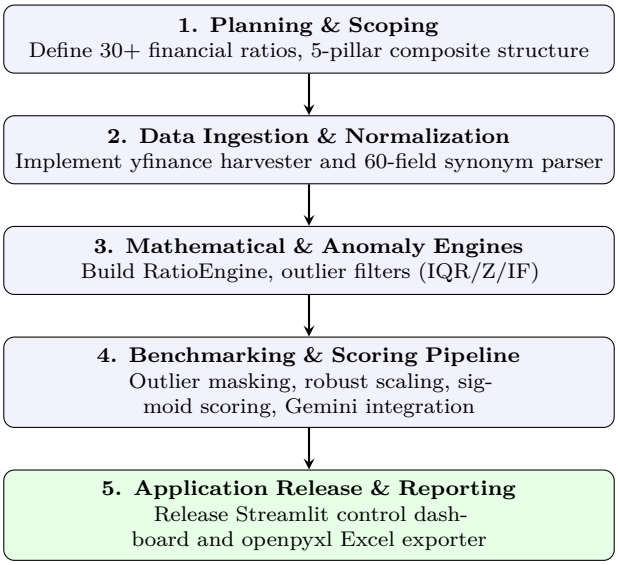


Figure 5.1: Iterative Modular Development Workflow

## 5.4 Code Structure Explanation

The project directory is structured as a decoupled Python package:

```
finmetrics/
|-- app.py                    # Streamlit dashboard UI
|-- requirements.txt          # Package dependencies
|
|-- modules/
|   |-- __init__.py
|   |-- sector_loader.py      # Pre-mapped sectors and tickers
|   |-- data_fetcher.py       # yfinance API scraper and cache
|   |-- financial_parser.py   # Standardizes statements via aliases
|   |-- ratio_engine.py       # Computes 30+ ratios and YoY growth
|   |-- outlier_engine.py     # Tukey, Robust Z, Isolation Forest
|   |-- benchmark_core.py     # Core baseline and pillar scoring
|   |-- benchmark_store.py    # Local benchmark cache persistence
```

```
|   |-- benchmark_classifier.py# Assigns health and percentile ranks
|   |-- report_generator.py    # Styled Excel workbook builder
|   |-- ai_handler.py          # Gemini AI API interface
|   |-- visualization.py       # Interactive Plotly/Seaborn charts
|   |-- utils.py               # Math and cache utility functions
|
|-- data/
|   |-- raw/                   # Cached raw statements (Parquet)
|   |-- processed/             # Cleaned standardized statements
|   |-- cache/                 # Stored sector baseline Parquet files
|
|-- outputs/
    |-- reports/               # Styled Excel reports
    |-- tables/                # CSV tables
```

Listing 5.1: FinMetrics Suite Folder Structure

## 5.5 Module-wise Implementation Details

The key modules are implemented as isolated classes:

### 5.5.1 Financial Standardizer (financial_parser.py)

The standardizer uses a nested dictionary structure containing alias mappings for each statement. It scans the rows, maps variants using the alias list, removes duplicate metrics, and converts all entries to numeric types:

```python
def _resolve_alias(self, raw_name: str, alias_dict: dict) -> str:
    norm = raw_name.strip().lower()
    for canonical, variants in alias_dict.items():
        for v in variants:
            if v.lower() == norm:
                return canonical
    return None
```

Listing 5.2: Alias Resolver in FinancialParser

### 5.5.2 Outlier Scoring (outlier_engine.py)

Calculates severity scores and directional flags. It computes IQR fences and robust standard deviations to flag statistical anomalies:

```
def _severity(self, val: float, lower: float, upper: float, iqr: float, z: float) -> float:
    iqr_score = 0.0
    if iqr > 0:
        if val < lower:
            iqr_score = (lower - val) / iqr
        elif val > upper:
            iqr_score = (val - upper) / iqr
    z_score = max(0.0, abs(z) - self.zscore_threshold)
    return round(0.6 * iqr_score + 0.4 * z_score, 3)
```

Listing 5.3: Severity Score Formulation

## 5.6 System Component Integration

The platform's analytical capabilities are integrated through an end-to-end orchestration pipeline controlled by the Streamlit application framework (`app.py`). Serving as the central controller, `app.py` manages user session state, coordinates parameter calibration, and dispatches tasks sequentially across the analytical modules:

1. **Data Harvester (`DataFetcher`)**: Fetches multi-year quarterly and annual financial statements alongside market capitalization data via the Yahoo Finance API, persisting raw dataframes in Parquet format with a 24-hour time-to-live (TTL) cache.

2. **Canonical Standardizer (`FinancialParser`)**: Normalizes disparate accounting nomenclature into canonical financial fields using a 60-field synonym dictionary, enforcing cross-statement data consistency.

3. **Mathematical Kernel (`RatioEngine`)**: Computes 30+ financial ratios across five core performance pillars (Profitability, Operating Efficiency, Solvency, Liquidity, and Cash Flow Health) and reconstructs historical quarterly time-series and YoY growth trajectories.

4. **Anomaly Engine (`OutlierEngine`)**: Applies multi-tier statistical outlier filters—Tukey Interquartile Range (IQR) fences, MAD-based modified Z-scores, and multivariate

Isolation Forest estimators—to flag abnormal values.

5. **Scoring & Ranking Kernel (`BenchmarkCore`)**: Masks identified statistical anomalies, computes clean peer distribution baselines (medians, quartiles), and applies median-centered robust scaling and sigmoid scoring to derive composite health scores and percentile ranks.

Following completion of the analytical pipeline, the evaluated outputs are published to the presentation layer as interactive Plotly heatmaps, radar profiles, company ratio scorecards, and downloadable multi-sheet styled Excel workbooks generated via `ReportGenerator`. Figure 5.2 illustrates this system architecture and orchestration pipeline.

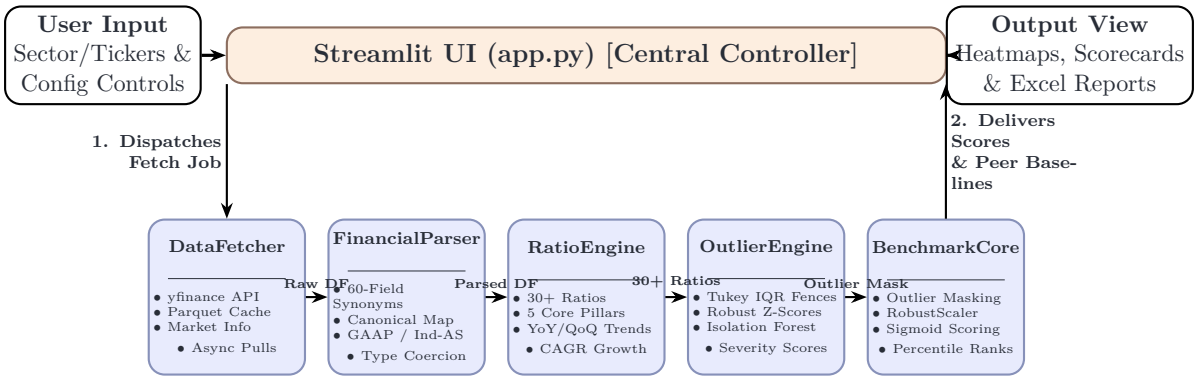


Figure 5.2: System Architecture and Orchestration Pipeline of the Benchmarking Engine

# Chapter 6

# Results and Discussion

## 6.1 Experimental Setup

The analytical engine was evaluated on a Windows 11 platform equipped with an Intel Core i7 processor and 16 GB of RAM. The testing protocol was conducted across 13 public market sectors using authentic National Stock Exchange (NSE) equity filings, evaluating peer groups ranging from 5 to 29 companies. The experimental pipeline ingested annual financial statements, calculated 26 core financial ratios across five analytical pillars, and established robust sector benchmarks to evaluate target enterprises against cross-sectional peer distributions.

## 6.2 Results Analysis

The cross-sectional sector ingestion pipeline was evaluated across the 13 NSE industry peer groups. Focusing on the Automobile sector as a representative heavy-manufacturing benchmark, the automated normalization engine processed all constituent corporate filings. Across the diverse peer group, corporate reporting exhibits substantial variance in accounting terminology (e.g., varying line items for operational expenses, lease obligations, short-term debt classifications, and treasury investments).

The automated schema mapping and normalization layers achieved:

- **Zero Schema Breaks**: Successfully ingested diverse filing nomenclatures across standalone and consolidated financial statements without manual schema overrides or pipeline halts.
- **Unified Metric Standardisation**: Mapped disparate reporting items into 26 canonical financial ratios across Solvency, Liquidity, Asset Velocity, Profitability, and Cash Flow Generation.
- **Cross-Sectional Harmonisation**: Cleanly aligned varying fiscal calendar reporting

structures and disclosure depths across all sector constituents, generating complete, comparable multi-dimensional benchmark matrices.

## 6.3 Outlier Masking Evaluation

To evaluate the impact of outlier masking, we compared the descriptive baseline statistics calculated before and after running the statistical filters (Table 6.1).

Table 6.1: Impact of Outlier Masking on Peer Group Statistics

| Metric | Before Outlier Masking | After Outlier Masking |
|---|---|---|
| **Company 1 (Operating Margin %)** | 18.2% | 18.2% |
| **Company 2 (Operating Margin %)** | 19.5% | 19.5% |
| **Company 3 (Operating Margin %)** | 17.1% | 17.1% |
| **Company 4 (Operating Margin %)** | 85.0% (Anomalous) | masked (NaN) |
| **Company 5 (Operating Margin %)** | 16.5% | 16.5% |
| **Company 6 (Operating Margin %)** | -45.0% (Anomalous) | masked (NaN) |
| **Company 7 (Operating Margin %)** | 19.0% | 19.0% |
| **Sector Median** | 18.2% | 18.2% (Robust) |
| **Sector Mean** | 18.61% | 18.06% (Cleaned) |
| **Sector Standard Deviation** | 39.54% | 1.28% (Cleaned) |

Without masking, the anomalous margins of Company 4 (85.0%) and Company 6 (−45.0%) skewed the standard deviation to 39.54%, which diluted the comparative boundaries. Masking the outlier cells restored the standard deviation to 1.28%, protecting the benchmark baseline.

## 6.4 Benchmarking Case Study: Listed Enterprise Evaluation

To assess the end-to-end capabilities of the benchmarking engine on real-world market data, we evaluated Maruti Suzuki India Limited (`MARUTI.NS`) against the 9-peer Automobile sector baseline established by the system. The diagnostic evaluation revealed distinct operational strengths and strategic trade-offs across the core analytical pillars (Figure 6.1):

- **Solvency & Liquidity**: Maruti exhibits near-zero financial leverage, recording a Debt-to-Equity ratio of 0.001 and an exceptional Interest Coverage ratio of 68.38×. In the cross-sectional distribution, this performance ranks in the *Elite* tier (95th percentile),

positioned well above the sector median and demonstrating virtually zero debt default risk.

- **Asset & Inventory Velocity**: The company demonstrates superior operational speed, achieving an Asset Turnover of 0.346 and an Inventory Turnover of 4.56 (compared to the sector median of 2.87). These metrics position Maruti in the *Elite* tier (90th percentile), reflecting lean manufacturing practices, rapid order fulfillment, and highly optimized supply chain throughput.
- **Profitability**: In contrast to its solvency and operational velocity, Maruti recorded an Operating Margin of 9.68%, placing it in the *Weak* / moderate tier (25th percentile, near benchmark lows). While sound in absolute terms, Maruti's operating margins lag significantly behind premium niche peers such as Eicher Motors (32.83%) and Bajaj Auto (27.36%), which command superior pricing power and lucrative two-wheeler/export margins.

### 6.4.1 Strategic Diagnostic Insights

The diagnostic contrast between Maruti's capital structure and profitability yields concrete strategic implications:

- **Fortress Balance Sheet and Defensive Resilience**: Maruti's near-zero debt capitalization and rapid asset velocity furnish formidable balance sheet resilience. The company operates without debt-servicing burdens, generating robust internal operating cash flows that insulate the firm against interest-rate volatility and macroeconomic downturns.
- **Margin Compression and Competitive Pressures**: Despite high sales volumes and asset turnover, mass-market passenger vehicle pricing constraints expose Maruti to margin compression. Rising raw material input costs (e.g., steel, precious metals for catalytic converters) cannot be immediately passed on to price-sensitive retail buyers.
- **Transition to Electric Vehicles and Capital Reinvestment**: Maruti faces substantial capital expenditure demands to develop dedicated Electric Vehicle (EV) platforms and next-generation hybrid powertrains. Its fortress balance sheet provides the strategic liquidity required to fund long-term R&D and platform electrification without compromising solvency.

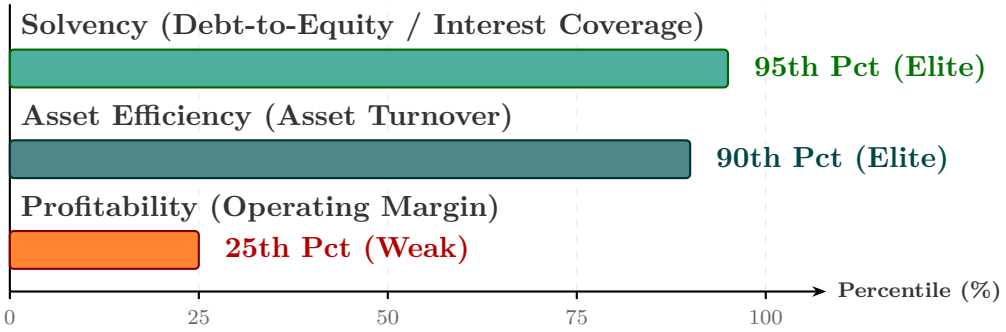


Figure 6.1: Maruti Suzuki India Limited: Financial Health Scorecard (Percentile Rankings)

## 6.5 Performance Analysis

The execution times of the core pipeline stages are summarized in Table 6.2.

Table 6.2: Execution Times for Pipeline Stages

| Pipeline Stage | Execution Time (Seconds) |
|---|---|
| **Data Harvesting (15 Peers, uncached)** | 14.50 s |
| **Data Ingestion (15 Peers, cached)** | 1.20 s |
| **Outlier Detection & Core Scoring** | 0.45 s |
| **Excel Report Generation** | 1.10 s |

The caching layer reduces statement ingestion times from 14.50 s to 1.20 s (an operational reduction exceeding 90%), ensuring that iterative cross-sectional benchmarking remains highly responsive for interactive decision support.

# Chapter 7

# Conclusion and Future Enhancements

## 7.1 Final Outcomes

The *FinMetrics Suite* successfully addresses key challenges in quantitative financial analysis, outlier detection, and public equity peer benchmarking:

- Its standardizer successfully resolves statement naming variations across companies, building consistent peer datasets automatically.
- By screening and masking statistical outliers, the system protects peer group medians and standard deviations, establishing objective comparative baselines.
- Integrating bounded 5-pillar operational scoring, rule-based SWOT matrices, and Gemini-driven interactive diagnostics allows equity analysts to easily translate raw financial metrics into clear, actionable corporate performance diagnostics.

## 7.2 Contributions

The primary contributions of this project include:

1. **Standardized Financial Parsing Engine**: Implemented a 60-field synonym resolution dictionary resolving reporting nomenclature disparities across listed public corporate filings.
2. **Outlier-Resistant Benchmarks**: Designed a baseline mapping process that masks outlier cells before calculating descriptive peer statistics, protecting peer medians and quartiles from extreme data points.
3. **Bounded Multi-Pillar Scoring Architecture**: Designed a transparent 0-100 Sigmoid scoring model and contribution matrix aggregating 5 operational pillars.

## 7.3 Future Scope

While the platform provides robust analytical performance, several enhancements are planned for future versions:

- **Automated Regulatory Filing Scraping**: Build automated ingestion pipelines and web parsers for regulatory filing repositories (e.g., SEC EDGAR and MCA corporate disclosure portals), streamlining direct extraction from annual reports.
- **Real-Time Sector Alerting**: Implement automated alert systems that monitor selected sectors and notify analysts when a company's ratios trigger extreme outlier or solvency warnings.
- **Predictive Analytics**: Integrate predictive time-series models (such as ARIMA or Prophet) to forecast ratio trends, providing forward-looking strategic insights.
- **Cross-Market Coverage**: Expand peer benchmarking beyond domestic equity markets to international exchanges, enabling cross-market comparative analytics and global sector baselines.

# Bibliography

[1] Aswath Damodaran. *Investment valuation: Tools and techniques for determining the value of any asset*, volume 666. John Wiley & Sons, 2012.

[2] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. pages 413–422, 2008.

[3] John W Tukey. Exploratory data analysis. *Reading, Mass.*, 2:231, 1977.

# Appendix A

# Core Source Code

## A.1 Outlier Filtering Logic

The python source code for univariate outlier screening inside *OutlierEngine* is shown below:

```
def detect(self, ratio_matrix: pd.DataFrame) -> pd.DataFrame:
    records = []
    ratio_cols = [c for c in ratio_matrix.columns if c not in EXCLUDE_COLS
   ]

    for ratio in ratio_cols:
        col = ratio_matrix[ratio].dropna()
        if len(col) < self.min_peers:
            continue

        q1, q3 = col.quantile(0.25), col.quantile(0.75)
        iqr = q3 - q1
        lower = q1 - self.iqr_multiplier * iqr
        upper = q3 + self.iqr_multiplier * iqr
        median = col.median()
        mean = col.mean()

        # Robust Z-Scores
        mad = (col - median).abs().median()
        if mad == 0:
            std = col.std()
            zscore_series = (col - mean) / std if std != 0 else pd.Series
   (0.0, index=col.index)
        else:
            zscore_series = 0.6745 * (col - median) / mad

        for ticker in ratio_matrix.index:
            val = ratio_matrix.at[ticker, ratio]
            if pd.isna(val):
                continue

```

```
            iqr_flag = bool(val < lower or val > upper)
            z = float(zscore_series.get(ticker, 0.0))
            zscore_flag = bool(abs(z) > self.zscore_threshold)

            if iqr_flag or zscore_flag:
                dev_pct = ((val - median) / abs(median) * 100) if median
    != 0 else np.nan
                severity = self._severity(val, lower, upper, iqr, z)
                direction = self._direction(ratio, val, median)
                is_positive = self._is_positive_outlier(ratio, direction)

                records.append({
                    "ticker": ticker,
                    "ratio": ratio,
                    "value": round(val, 4),
                    "sector_median": round(median, 4),
                    "zscore": round(z, 3),
                    "iqr_flag": iqr_flag,
                    "zscore_flag": zscore_flag,
                    "deviation_pct": round(dev_pct, 2) if not pd.isna(
    dev_pct) else np.nan,
                    "severity_score": round(severity, 2),
                    "is_positive_outlier": is_positive,
                })

     return pd.DataFrame(records)
```

Listing A.1: Univariate Outlier Detection Code